

A Graph *b*-coloring Framework for Data Clustering

Haytham Elghazel · Hamamache Kheddouci ·
Véronique Deslandres · Alain Dussauchoy

Received: 15 January 2007 / Accepted: 3 October 2008 /
Published online: 26 November 2008
© Springer Science + Business Media B.V. 2008

Abstract The graph *b-coloring* is an interesting technique that can be applied to various domains. The proper *b-coloring* problem is the assignment of colors (classes) to the vertices of one graph so that no two adjacent vertices have the same color, and for each color class there exists at least one *dominating vertex* which is adjacent (*dissimilar*) to all other color classes. This paper presents a new graph *b*-coloring framework for clustering heterogeneous objects into groups. A number of cluster validity indices are also reviewed. Such indices can be used for automatically determining the optimal partition. The proposed approach has interesting properties and gives good results on benchmark data set as well as on real medical data set.

Keywords Clustering · Graph *b*-coloring · Cluster validity indices

1 Introduction

Clustering, or *unsupervised classification*, is a fundamental data mining process that aims to classify observations into groups or categories, such that all objects in the same group are *similar* to each other, while *dissimilar* from objects in other groups.

H. Elghazel · H. Kheddouci · V. Deslandres · A. Dussauchoy
Université de Lyon, Lyon, 69003, France

H. Elghazel (✉) · H. Kheddouci · V. Deslandres · A. Dussauchoy
Université Lyon 1, LIESP, Villeurbanne, 69622, France
e-mail: elghazel@bat710.univ-lyon1.fr

H. Kheddouci
e-mail: hkheddou@bat710.univ-lyon1.fr

V. Deslandres
e-mail: deslandres@bat710.univ-lyon1.fr

A. Dussauchoy
e-mail: dussauchoy@bat710.univ-lyon1.fr

Clustering plays an important role in data mining applications such as Web analysis, information retrieval, medical diagnosis, and many other domains.

The similarity and dissimilarity between objects are calculated using metric or semi-metric functions applied to the attribute values characterizing objects.

Let $V = \{v_1, v_2, \dots, v_n\}$ be the object set to be clustered. Each object is represented by a mixture of m symbolic and/or classical attributes (a set of relevant features of the objects) denoted a_f ($f \in \{1 \dots m\}$). Therefore, the object v_i is described by an m -dimensional vector $(a_{i,1}, a_{i,2}, \dots, a_{i,m})$. Classical attributes are defined by either quantitative or qualitative values while symbolic attributes can be either a set of values or intervals. Usually, the attributes characterizing objects are standardized in order to give equal weight to all of them. The dissimilarity measure between objects v_i and v_j is given by the *normalized generalized Minkowski* distance of order α ($\alpha \geq 1$) [1].

$$d_{i,j} = D(v_i, v_j) = \left(\sum_{f=1}^m \left(\frac{g_f(a_{i,f}, a_{j,f})}{m_f} \right)^\alpha \right)^{\frac{1}{\alpha}} \quad (1)$$

When $\alpha = 1$, the distance is exactly the same as the *City Block distance*. When $\alpha = 2$, the distance metric is exactly the same as the *Euclidian distance*. As α increases, the distance tends towards a *Chebyshev* one.

m_f is the normalized coefficient for the attribute a_f .

g_f is the *comparative dissimilarity function* between the two attribute values $a_{i,f}$ and $a_{j,f}$ corresponding respectively to the objects v_i and v_j .

This paper is not intended to focus on dissimilarity measures. For issues related to comparative functions, the reader can refer to [2].

The main goal of this work is to identify a partition of symbolic objects grouped together into clusters, without requiring any input parameters, and where optimal cluster separation is obtained simultaneously with optimal cluster cohesion.

The rest of the paper is organized as follows: previous related works are discussed in Section 2, and in Section 3, the b-coloring method is introduced. Section 4 describes the new cluster partitioning algorithm. Section 5 is devoted to the cluster validity indices. Some experiments using relevant benchmarks are presented in Section 6. In Section 7, we also discuss experiments conducted on a real medical data set which represents patient hospital stays. The experiments show that the algorithm is relevant for discovering a new typology of hospital stays with compact and well-separated groups. This conclusion is the result of the validation stage of the method conducted with experts of the domain. We conclude in Section 8 with useful propositions of extensions.

2 Related Work

Due to its wide applicability, many individuals have made substantial contributions to the field of cluster analysis. Jain et al. [3] survey many of these efforts. Clustering

algorithms are generally based on two popular clustering techniques, namely *partitioning* and *hierarchical* clustering. In the following, we give an overview of both techniques.

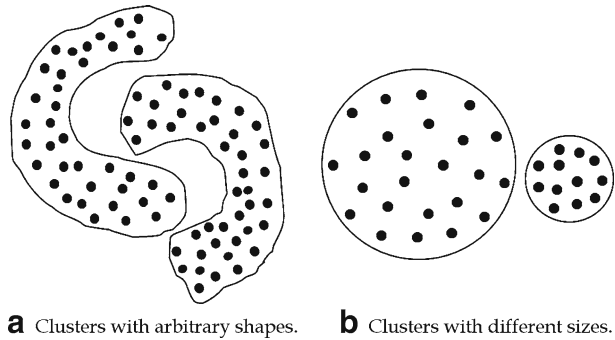
The *hierarchical clustering algorithms* build a cluster hierarchy or, in other words, a tree of clusters, (*dendrogram*) whose leaves are the data points and whose internal nodes represent nested clusters of various sizes. Hierarchical clustering methods can be further subdivided into *agglomerative* and *divisive*. Given n objects to be clustered, the agglomerative (*bottom-up*) methods begin with n clusters (*i.e.*, all objects are apart). In each step, the two most similar clusters are identified and merged. This process continues until all objects are clustered into one group. On the other hand, the divisive (*top-down*) methods start from one cluster containing the n objects and split it until n clusters are obtained. The Agglomerative Clustering Algorithms proposed in the literature differ in the way the similarity of clusters is calculated as well as in the linkage-metric used (*single-link*, *complete-link* or *average-link*). CURE [4] and Chameleon [5] are examples of hierarchical clustering algorithms. While hierarchical methods represent major classes of clustering techniques and have been successfully applied in several domains, they suffer from the well-known weakness, that is, they can never undo what was done previously. Once two objects have been merged by an agglomerative algorithm, they will always remain in the same cluster; if a divisive method separates two objects, they will never belong to the same cluster.

In contrast, given the number k of partitions to be found, *partitioning clustering algorithms* try to find the best k partitions of the n objects. The partitioning algorithm typically starts with an initial partition of a data set, and then uses an iterative control strategy to optimize an objective function. It is very often the case that the k clusters found by a partitioning method are of higher quality (*i.e.*, more similar) than the k clusters produced by a hierarchical method [6]. Owing to this property, developing partitioning methods has been one of the main focuses of cluster analysis research. Indeed, many partitioning methods have been developed, some based on *k-means* [7], some on *k-medoid* [6], etc. The most popular partitioning algorithm used in scientific and industrial applications is *k-means*. In *k-means*, each cluster is represented by its centroid, which is the average of the data vectors in a cluster. The *k-means* attempts to minimize E , which is the sum of squared distances to the cluster centroids. Although *k-means* often finds the local optimum, it works well when objects within each cluster are quite close to each other while the objects from different clusters are not. It also displays some limitations on attribute types (it does not work well with symbolic attributes) and it is very sensitive to (1) the selection of the initial partition and (2) to the presence of outliers.

The *k-medoid* algorithm overcomes some of the above objections. Here, each cluster is represented by its medoid which is the most centrally located object. The *k-medoid* is a two-step procedure. First, it determines the k representative objects which minimize the objective function E . Then, it assigns each object to the closest cluster (whose medoid is the closest to the considered object). Unlike *k-means*, the *k-medoid* methods present no limitations on attribute types and are robust in the presence of outliers.

However, a main problem of both of these schemes is that they use a single point as the representative of a cluster. Since the purpose of these algorithms is to find clusters

Fig. 1 Data sets on which centroid and medoid approaches fail (**a**, **b**)



that minimize E , they fail for data where some objects are closer to the centroid of another cluster than to the centroid of their own cluster. Thereby, they cannot easily capture the cluster with arbitrary shapes (c.f. Fig. 1a) or with a large variation in cluster sizes (c.f. Fig. 1b) [5].

When the distances (dissimilarities) among all pairs of data $\{v_1, v_2, \dots, v_n\}$ are specified, these can be summarized as a weighed dissimilarity matrix D in which each element $D(v_i, v_j)$ stores the corresponding dissimilarity. Based on D , the data can also be conceived as a weighted linkage graph $G(V, E)$, where $V = \{v_1, v_2, \dots, v_n\}$ is the vertex set which correspond to the data, and $E = V \times V$ is the edge set which correspond to a pair of vertices (v_i, v_j) weighted by the dissimilarity $D(v_i, v_j)$. In that framework, additional clustering mechanisms are based on graph theoretical techniques [8]. They basically consist in finding combinatorial structures within the similarity/dissimilarity graph, such as spanning tree¹ [9], connected components² [10, 11], minimum cut [12, 13], or maximal complete subgraphs³ [14].

The best-known graph-theoretic divisive clustering algorithm (the *single-link* algorithm) is based on construction of the *minimal spanning tree* (MST) of the data [9], and then deleting the MST edges with the largest lengths to generate a hierarchy of *single-link* clusters called *dendrogram*. Some authors have investigated the use of vertex coloring in spanning tree for hierarchical classification purpose. By considering the *diameter* of one cluster as the largest distance between two objects belonging to the cluster, Guénoche et al. have proposed in [15] a divisive classification method based on dissimilarity tables. They define an iterative algorithm which consists, at each step, in finding a partition by subdividing the cluster with the largest *diameter* into two clusters in order to obtain a minimal *diameter* partition (the *diameter* of one partition is defined by its largest *cluster diameter* value). The subdivision is obtained by a *2-coloring* of the vertices of the *maximum spanning tree* built from the dissimilarity table. The derived classification structure is a hierarchy.

¹A spanning tree of a graph G is a subgraph which is a tree without loops and connects all the vertices of G . A minimum spanning tree is a spanning tree with the minimum dissimilarity weight (sum of weights of the edges).

²A connected component H of a graph G is an induced subgraph $H \subseteq G$ such that the nodes in H are mutually reachable by proceeding along the edges of the graph.

³A complete subgraph namely clique is a subgraph in which all pairs of nodes are connected by an edge. Such a complete subgraph is maximal when it is not contained in any other complete subgraph.

The original suggestion to use connected components for discovering clusters was made by Matula [10, 11] in the seventies. He observed how useful could be connectivity analysis in similarity graphs for clustering purpose. Based on *edge-connectivity* of graph -which is the minimum number of edges whose removal disconnects the graph-, and the *cohesiveness* function -defined for every vertex and edge of a graph G to be the maximum edge-connectivity of any subgraph containing that element-, Matula [10] suggested identifying as clusters the maximal k -connected subgraphs of G by deleting all elements in G of cohesiveness less than k (where k is defined beforehand). The main problem in this approach is that different real clusters may have different connectivity values. As a solution, Matula then proposed in [11] to define as clusters maximal k -connected subgraphs for any k which do not contain a subcomponent with higher connectivity. With this choice, the risk is to split some real clusters which have many highly cohesive parts [16].

Additionally, Hartuv and Shamir [16] developed a *Highly Connected Subgraphs* (HCS) algorithm for cluster analysis based on *connected components concept*. They propose to use the similarity matrix between data set to form one *threshold graph* in which vertices correspond to elements, and edges connect elements with similarity values above a given threshold θ . In that graph, they define clusters as a *highly connected subgraphs* whose edge connectivity exceeds half the number of vertices. Such subgraphs are conceived using minimum cut algorithms. They proved that the resulting partition exhibits several desirable cluster properties: the generated clusters from HCS have a *diameter*⁴ at most two which is a strong indication of homogeneity as any two vertices are either adjacent or share one or more common neighbours. The authors also showed that this property is not verified by both Matula [11] and Wu and Leahy (presented below) [13] clustering algorithms.

On the other hand, Wu and Leahy [13] proposed using the *minimum cuts* in weighted similarity graphs for cluster analysis. In such a graph $G(V, E)$, a cut is a set of edges whose removal disconnects the graph by dividing its vertex set V in two parts V_1 and V_2 such that $V_1 \cup V_2 = V$ and $V_1 \cap V_2 = \emptyset$. The *cut cost* is the value of total edge weight (similarity) connecting the two parts and the optimal bipartitioning of G is the one that minimizes this cut value. Based on this minimum cut criterion and using the *Gomory and Hu algorithm* [17], Wu and Leahy propose to compute the $k - 1$ smallest cuts in G in order to partition it into k -subgraphs (k is defined in advance) and they show its effectiveness to produce good segmentation on some images. However, as Wu and Leahy also noticed in their work, the minimum cut criterion often favors cutting using a minimum edge number and causes an unbalanced partition; it may cut a portion of a graph with a small number of vertices. This is generally not desirable for cluster analysis. To weaken the edge number effect and avoid partitioning out a small part of a graph by using edge-cut alone, many graph partitioning criteria were proposed, such as *normalized cut* [12].

Today the most widespread graph-theoretic clustering approaches consist in finding *complete subgraphs* derived from an input *threshold graph*. Since *complete subgraphs* (*cliques*) are based on highly connected graph components, such structure

⁴In this case, the *diameter* of one cluster C is the longest distance between any two vertices in C , where the distance between two vertices is the minimum length of a path joining them.

are generally considered to provide highly homogenous groups. Several works exploring this idea are reported in the literature. In [18], Kuhns was the first to define the maximal complete subgraph of a graph as one cluster, but he did not provide experimental results. Gotlieb and Kumar [14] also use the concept of *maximal complete subgraphs* for defining clusters. They focused on unweighted graphs derived from one similarity graph by the mean of some threshold operations. The *maximal complete subgraphs* study is likewise related to the vertex coloring of graphs. Indeed, Hansen and Delattre [19] reduced the partitioning problem into k classes with a minimal diameter,⁵ to the minimal coloring problem of a *superior threshold graph* in which vertices correspond to elements and edges correspond to dissimilarities between two elements which is higher than a given threshold value θ . In such a graph, each color corresponds to one class and the number of colors is minimal. Unfortunately, while these methods based on finding complete subgraphs tends to build a partition of the data set with effectively compact clusters, they does not give any importance to the *cluster-separation*.

In this paper, we continue the challenge to mix the *complete subgraph* structures and the clustering problem. The main objective is to ensure a high *cohesion* within the generated clusters and a high *cluster separation*. The clustering visualization is given using a new coloring method, namely the *b-coloring* of graphs. It allows building a partition of the data set (*classical* or *symbolic*) where the number of clusters is not set in advance. Indeed, the purpose of this approach is to use dissimilarities between every pair of objects (not only with the representative of the cluster) to find a cluster partition where internal cluster cohesion and separation among clusters are simultaneously effective. This approach exhibits more important features: (1) it responds to the problem of single-point representation of clusters by using the topological relations between objects when building clusters: all data objects participate in this phase, (2) it is robust in outlier detection and consideration, (3) it can accommodate all types of data as long as a dissimilarity table can be constructed, and (4) it provides each cluster with at least one *dominant object* which can be used for representation purposes, and which guarantees the disparity between clusters. As the approach is an iterative procedure which operates upon different threshold graphs, a cluster validity index is proposed in order to determine the optimal partition. It is designed to identify a cluster partition of data in compact and well separated groups.

An additional advantage of this *graph-based clustering* approach is its ability to being extended to an *online version* in order to process instances as they are added (*new data*) or deleted (*outmoded* or *inefficient data*) from the data collection [20]. It consists in updating clusters without having to frequently performing a complete re-clustering. The basic idea is, once the optimal partition returned from the *b-coloring based-clustering algorithm*, working to assign new instances to their respective clusters (*i.e.* appropriate color) or to rearrange the partition when existing instances leave the system. This is constrained to incrementally maintain the *b-coloring* of G and the clustering performances in terms of quality and runtime.

⁵In this case, the *diameter* of one cluster is the largest dissimilarity between two objects belonging to the same cluster.

3 Graph b-coloring

Let $G(V, E)$ be an undirected, connected and simple graph with a vertex set $V = \{v_1, v_2, \dots, v_n\}$ and an edge set E . The b -coloring of G is a vertex coloring function $c : V \rightarrow Z^+$ such that:

- For each pair of adjacent vertices $(v_i, v_j) \in E$, $c(v_i) \neq c(v_j)$ (proper coloring),
- In each color class, there exists at least one vertex having neighbors in all other color classes. Such a vertex is called a *dominating vertex*. A color which has a dominating vertex is called a *dominating color*.

We call the b -chromatic number $\varphi(G)$ of a graph G the largest number k such that G has a b -coloring with k colors.

In contrast to a *minimum coloring* of the vertices of graph G , a b -coloring is a *maximum coloring* under property and dominance constraints.

The number of neighbors of one vertex is called degree of the vertex and the maximum degree of the graph G , denoted by $\Delta(G)$, is the largest degree over all vertices of G .

Irving and Manlove [21] introduce the concept of *graph b-coloring* in 1999. They prove the following result:

Proposition 1 *Let G be a graph and $\chi(G)$ be the chromatic number which is the minimum number of colors required for a proper coloring of G . Then we have:*

$$\chi(G) \leq \varphi(G) \leq \Delta(G) + 1 \quad (2)$$

Proof As the b -coloring of G is a proper coloring so we have $\chi(G) \leq \varphi(G)$. It is easy to see that any vertex of degree $\Delta(G)$ can admit at most $\Delta(G)$ neighbor colors and takes a color $\Delta(G) + 1$. Therefore, $\varphi(G) \leq \Delta(G) + 1$. $\square$

Irving and Manlove have also shown that finding the b -chromatic number $\varphi(G)$ for any graph is a *NP-hard* problem. Several authors have investigated this parameter for particular classes of graphs like trees [21], power graphs [22] and Cartesian products of graph [23]. More recently, Effantin and Kheddouci [24] give a distributed algorithm to construct a b -coloring of graph. For more details, in [25], the author proposes a large survey on this parameter.

If G is a non-connected graph with connected components $C_1, C_2, \dots, C_p$, we have [23]:

$$\varphi(G) \geq \max\{\varphi(C_i), 1 \leq i \leq p\} \quad (3)$$

4 Clustering Using Graph b-coloring

In this section, a new clustering algorithm based on the b -coloring of graph is proposed. Consider the data to be clustered as an undirected edge-weighted graph $G(V, E)$, where $V = \{v_1, v_2, \dots, v_n\}$ is the vertex set and $E = V \times V$ is the edge set. Vertices in G correspond to data objects, edges represent neighborhood relationships, and edge-weights reflect dissimilarity between pairs of linked vertices. The graph G is traditionally represented with the corresponding weighted dissimilarity

matrix, which is the $n \times n$ symmetric matrix $D = \{d_{ij} | v_i, v_j \in V\}$. A common informal definition states that “a cluster is a set of entities which are *similar*, and entities from different clusters are not *similar*”. Hence, a cluster should satisfy two fundamental conditions: (1) it should have high internal homogeneity; (2) there should be high heterogeneity among entities from different clusters. These two conditions amount to saying that edges between two vertices within one cluster should be small weighted (denoting high similarity), and those between vertices from two clusters should be large weighted (weak similarity). The clustering problem is hence formulated as a graph b-coloring problem.

4.1 Construction of the Threshold Graph

Since the objects to be clustered are considered as an *undirected complete edge-weighted graph* where the vertex set is the set of data and the edge-weights set reflects the dissimilarity pairs of linked vertices, the b-coloring of such a complete graph is not interesting for the clustering problem. Indeed, each cluster (*color*) is assumed to contain one and only one object (*vertex*).

Consequently, our clustering approach requires to construct a *superior threshold graph*, which is a partial graph of the original one $G(V, E)$. Let $G_{>\theta}(V, E_{>\theta})$ be the superior threshold graph associated with threshold value θ chosen among the dissimilarity table D . In other words, $G_{>\theta}$ is given by $V = \{v_1, v_2, \dots, v_n\}$ as vertex set and $\{(v_i, v_j) | D(v_i, v_j) = d_{ij} > \theta\}$ as edge set. For an illustration purpose, Fig. 2 gives the superior threshold graph $G_{>0.15}$ ($\theta = 0.15$) for the following dissimilarity Table 1. In $G_{>0.15}$, edges are labeled with the corresponding dissimilarities.

Fig. 2 The superior threshold graph $G_{>0.15}$ ($\theta = 0.15$)

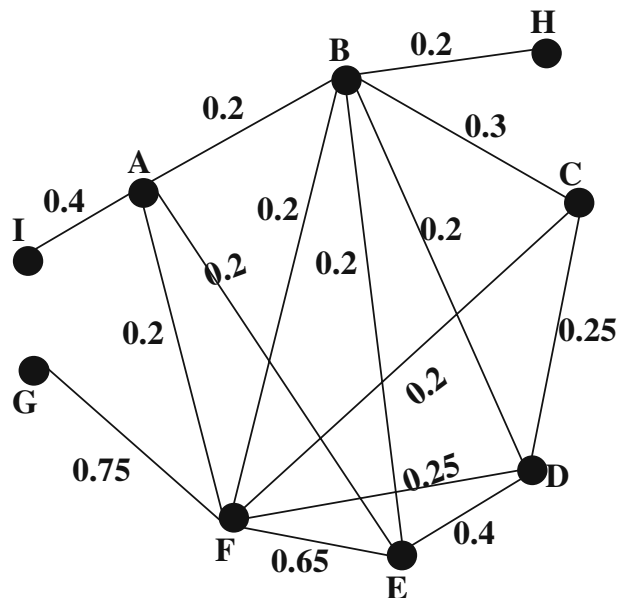


Table 1 A weighted dissimilarity table

v_i	A	B	C	D	E	F	G	H	I
A	0								
B	0.20	0							
C	0.10	0.30	0						
D	0.10	0.20	0.25	0					
E	0.20	0.20	0.10	0.40	0				
F	0.20	0.20	0.20	0.25	0.65	0			
G	0.15	0.10	0.15	0.10	0.10	0.75	0		
H	0.10	0.20	0.10	0.10	0.05	0.05	0.05		
I	0.40	0.075	0.15	0.15	0.15	0.15	0.15	0.15	0

In the following, recall that two vertices are adjacent (*resp.* no adjacent) means that the two instances (vertices) have a *dissimilarity degree* higher (*resp.* lower or equal) than the threshold value θ .

4.2 Algorithm

A new b-coloring algorithm, well suited for the cluster analysis problem, has been presented in [26]. The main idea of this paper is to apply this algorithm on the superior threshold graph $G_{>\theta}$. Indeed, the goal is to give an assignment of colors (classes) to the vertices of the graph $G_{>\theta}$ so that no two adjacent vertices (pair vertices whose dissimilarity is higher than the threshold θ) have the same color, and that for each color class, at least one *dominating vertex* is adjacent to at least one vertex of every other color sets.

A partition associated with the selected threshold value θ is then returned. The algorithm exhibits nice properties that are relevant for clustering, notably the *intracluster homogeneity* (tightness) and the *intercluster separation* as demonstrated below.

Notations Let $G(V, E_{>\theta})$ be the threshold graph, where $V = \{v_1, v_2, \dots, v_n\}$ is the vertex set and $E_{>\theta}$ the edge set. In the remaining of the paper, assuming that the vertices of G are colored, the notations used are the following:

- Δ : the maximum degree of G .
- $c(v_i)$: the color (integer value) of the vertex v_i in G .
- $N(v_i)$: the neighborhood of vertex v_i in G .
- $N_c(v_i)$: the neighborhood colors of vertex v_i .
- L : the color set used in the graph (one set of integer values).
- D_m : the set of colors which have dominating vertices.
- ND_m : the set of colors that have no dominating vertex.
- $dist(v_i, c)$: the distance between the vertex v_i and the color c which is defined by the *single-linkage distance* as the distance between v_i and the nearest vertex having color c :

$$dist(v_i, c) = \min\{d_{i,j} = D(v_i, v_j) | 1 \leq i \neq j \leq n \text{ and } c(v_j) = c\} \quad (4)$$

The data to be clustered are now depicted by a non-complete edge-weighted graph $G(V, E_{>\theta})$. The goal is to divide the vertex set V into a partition $P = \{C_1, C_2, \dots, C_k\}$ where for $\forall C_i, C_j \in P, C_i \cap C_j = \emptyset$ for $i \neq j$. (when the number

of clusters k is not pre-defined). The notation of $\mathbf{P}$ is used to both represent a set of clusters as well as a set of colors, because each cluster $C_i \in \mathbf{P}$ corresponds to a color in our b-coloring based clustering algorithm which consists of two steps: 1) initializing the colors of vertices with maximal colors, and 2) removing colors without any dominating vertex using a *greedy procedure*.

Step 1—Initial Coloring The following routines are proposed for the algorithm:

- $Update(N_c(v_i))$ is the method which updates the neighborhood colors of the vertex v_i when the color of at least one of its neighbors has changed.
- $Add_sort(o, \mathbf{S})$ is the method which adds the vertex o into the vertex set $\mathbf{S}$ by respecting the decreasing order of vertex degree.
- $Add(o, \mathbf{S})$ is the method which adds the vertex (*resp.* color) o into the vertex (*resp.* color) set $\mathbf{S}$.
- $Remove(o, \mathbf{S})$ is the method which removes the object o from the set $\mathbf{S}$.

Let us consider an additional notation $\mathbf{T}$ to denote the set of colored vertices which is sorted on descending order of vertex degrees. Initially, since vertices of G are not already colored, $\mathbf{T}$ is an empty set $\{\phi\}$ and will be updated as long as *Step 1* runs, as shown below.

The Algorithm 1 *Init-b-coloring* provides an initial configuration using the maximum number of colors available, *i.e.* $\Delta + 1$ (Eq. 2). It starts from one vertex of $\mathbf{V}$ which has the maximum degree Δ (let v be such a vertex). The algorithm puts $c(v) = 1$ and adds v to the set $\mathbf{T}$. Then Algorithm 1 tries to color remaining vertices according to the following principle: for each vertex v_i belonging to $\mathbf{T}$ (initially $\mathbf{T} = \{v\}$), a new color is assigned to each one of its neighbors v_j if it is not already colored and which will be then added to $\mathbf{T}$. In order to give a proper vertex-coloring of G with a maximum number of colors, the color of v_j should be different from those of its neighbors and -if possible- different from the one of v_i neighbors. If all existing colors in G do not satisfy both constraints, the chosen color for v_j must at least verify the first one (*i.e.* the color must be different from those of its neighbors, which gives what is called a *proper vertex-coloring* of G). For maximizing *intracluster homogeneity*, the sum of vertex dissimilarities within each cluster has to be minimized. So the color whose distance with v_j is minimal will be selected if there is a choice between many colors for v_j .

After the coloring of every neighbor v_j of the vertex v_i , the instructions (21, 22, 23) check if the color $c(v_i)$ is a new dominating color. After that, the vertex v_i is removed from $\mathbf{T}$.

A Working Example The Algorithm 1 performed on the previous threshold graph $G_{>0.15}$ starts with the vertex $\mathbf{B}$ ($\mathbf{T} = \{\mathbf{B}\}$) having a degree value of 6 (which is the maximum degree of the graph $G_{>0.15}$). It puts $c(\mathbf{B}) = 1$ and assigns a new color to each one of its neighbors. Consequently, it gives the colored graph in Fig. 3.

The set $\mathbf{T}$ contains now the objects $\{\mathbf{F}, \mathbf{A}, \mathbf{D}, \mathbf{E}, \mathbf{C}, \mathbf{H}\}$ which are sorted on descending order of their degree. Then Algorithm 1 tries to color the remaining vertices ($\mathbf{G}$ and $\mathbf{I}$). Given the vertex $\mathbf{F}$ from $\mathbf{T}$, a new color (**Color 7**) is assigned to the vertex $\mathbf{G}$ which is different from those of its neighborhood and the neighborhood of $\mathbf{F}$. Figure 4 gives the returned configuration.

Algorithm 1 Init-b-coloring**Require:** $G(V, E)$; //A graph with a set of vertices and a set of edges.

```

1: Select the vertex  $v$  with the highest degree;
2:  $c(v) = 1$ ;
3:  $Add(v, T)$ ;
4:  $L := \{1, 2, \dots, \Delta + 1\}$ ;
5: repeat
6:   Select  $v_i$  from  $T$ ;
7:    $M := N_c(v_i) \cup c(v_i)$ ;
8:   for each vertex  $v_j \in N(v_i)$  such that  $c(v_j) = \phi$  do
9:      $q := \min\{h | h > 0, h \notin M \text{ and } h \notin N_c(v_j)\}$ ;
10:    if ( $q < \Delta + 1$ ) then
11:       $c(v_j) := q$ ;
12:    else
13:       $H := \{h | h \in L \text{ and } h \notin N_c(v_j)\}$ ;
14:       $c(v_j) := \operatorname{argmin}_{h \in H}(\operatorname{dist}(v_j, h))$ ;
15:    end if
16:     $Add\_sort(v_j, T)$ ;
17:    for each vertex  $v_h \in N(v_j)$  do
18:       $Add(c(v_j), N_c(v_h))$ ;
19:    end for
20:  end for
21:  if ( $N_c(v_i) = L \setminus \{c(v_i)\}$ ) then
22:     $Add(c(v_i), D_m)$ ;
23:  end if
24:   $Remove(v_i, T)$ 
25: until  $T = \phi$ 

```

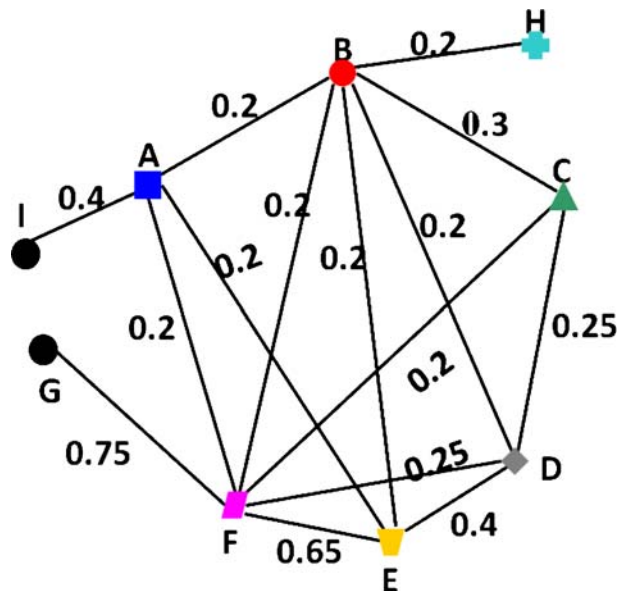
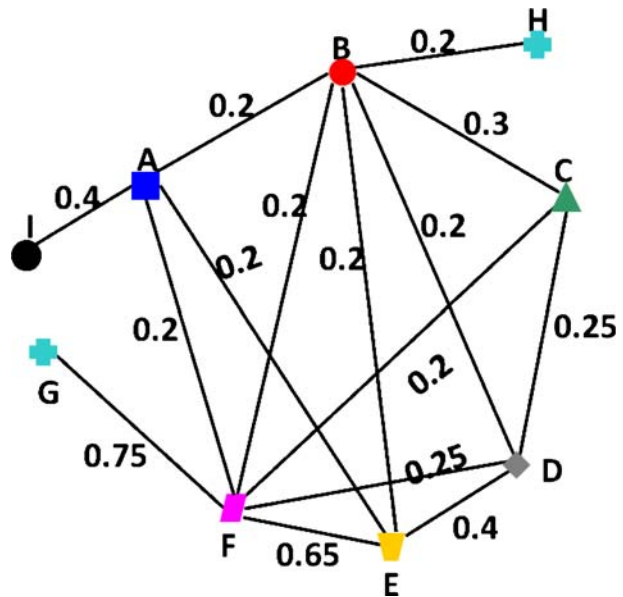
Fig. 3 Partially colored graph $G_{>0.15}$ with vertex **B**

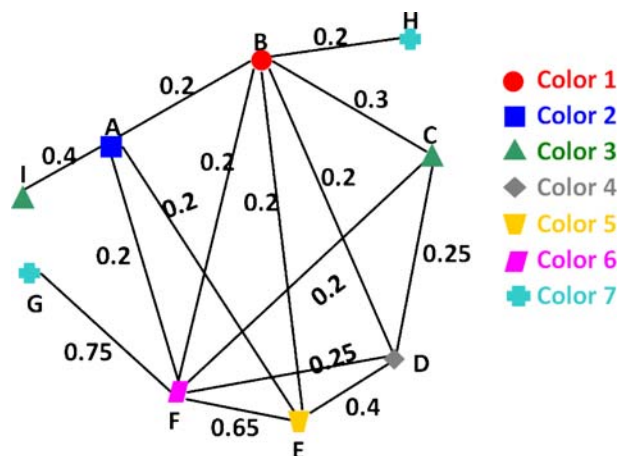
Fig. 4 Partially colored graph $G_{>0.15}$ with vertex **F**



As a result, T is given now by the set $\{A, D, E, C, H, G\}$. Considering the vertex **A** from T , a new color (**Color 3**) is assigned to the vertex **I** which is different from those of its neighborhood and the neighborhood of **A**. Thus, the initial configuration with the maximum number of colors available for the b-coloring (i.e. $\Delta + 1 = 7$) is returned in Fig. 5. Among the seven given colors, there are only two dominating colors (**Color 1** and **Color 6**) which have each one a dominating vertex (**B** for **Color 1** and **F** for **Color 6**) $\Rightarrow D_m = \{1, 6\}$.

Proposition 2 The Algorithm 1 runs in $O(n^2)$.

Fig. 5 Initial coloring of graph $G_{>0.15}$ with maximal colors



Proof Algorithm 1 is applied once for each colored vertex (at most n). First, the algorithm assigns a color to every uncolored neighbor (at most Δ). Each neighbor is then introduced in T by respecting the decreasing order of vertex degree (at most n : $O(n)$) and the change of color is propagated to its own neighbors (at most Δ). Therefore Algorithm 1 uses at most $(n \times \Delta \times (n + \Delta))$ instructions, and the complexity is $O(n^2)$. $\square$

Step 2—Towards the b-coloring of G After performing Algorithm 1, some assigned colors (the set L) remain without any dominating vertex. The objective of the Algorithm 2 is to find a b-coloring of graph G where all colors are dominating colors (i.e. $D_m = L$). The idea is the following: each non-dominating color l (i.e. $l \in ND_m$) can be changed (c.f. Proposition 3). In fact, after removing l from the graph G , for each vertex v_i colored with l (i.e. $c(v_i) = l$), a new color is assigned to v_i which is different from those of its neighborhood. As for *Step 1*, the color whose distance with v_i is minimal will be selected if there is a choice between many colors for v_i . Before starting again with another color $l' \in ND_m$, the Algorithm verifies if colors of ND_m have a dominating vertex (in such a case, these colors are added to the D_m set).

Once all the vertices v_i with the color q have been modified (re-colored), it must be verified using instructions (15, 16, 17) in Algorithm 2 if some non-dominating colors now become dominating colors.

Obviously, the coloring generated by Algorithm 2 is a *proper coloring*. This follows from the fact that when the color $c(v_i)$ is changed, the new color is selected to be different from the neighbor's colors of v_i .

Algorithm 2 reaches a fix point when the set of colors with no dominating vertices is an empty set. As a consequence, there exists at least one dominating vertex for each color class.

Algorithm 2 Find-b-coloring

```

1: Select the vertex  $v$  with the highest degree;
2:  $c(v) = 1$ ;
3:  $Add(v, T)$ ;
4:  $L := \{1, 2, \dots, \Delta + 1\}$ ;
5: repeat
6:    $l := \operatorname{argmax}_{h \in ND_m}$ ;
7:    $L := L \setminus \{l\}$ ;
8:    $ND_m := L - D_m$ ;
9:   for each vertex  $v_i$  such that  $c(v_i) = l$  do
10:     $H := \{h | h \in L \text{ and } h \notin N_c(v_i)\}$ ;
11:     $c(v_i) := \operatorname{argmin}_{h \in H} (dist(v_i, h))$ ;
12:   end for
13:   for each vertex  $v_j$  such that  $c(v_j) \in ND_m$  do
14:      $Update(N_c(v_j))$ 
15:     if  $(N_c(v_j) = L \setminus \{c(v_j)\})$  then
16:        $Add(c(v_j), D_m)$ ;
17:     end if
18:   end for
19: until  $ND_m = \phi$ 

```

Proposition 3 Every non-dominating color q can be easily changed.

Proof A non-dominating color q is a color without dominating vertices. Hence, for each vertex v colored with q , there exists at least one color c not adjacent to v . (otherwise q will be dominating) Therefore, the vertex v can always be re-colored with c . Consequently, the color q can be easily removed from G . $\square$

Proposition 4 The Algorithm 2 generates the b -coloring of any graph in $O(n\Delta^2)$.

Proof Algorithm 2 is applied for every color l without dominating vertices (at most Δ). The color l is removed, and every vertex v_i such that $c(v_i) = l$ (at most n) is recolored. Then, for every vertex v_j such that $c(v_j) \in ND_m$ (at most n), its neighborhood colors $N_c(v_j)$ is updated (at most Δ) and thus its $c(v_j)$ is tested to be a new dominating color. Therefore, Algorithm 2 uses at most $(\Delta \times (n + n + \Delta))$ instructions, and the complexity is $O(n\Delta^2)$. $\square$

A Working Example Let us consider the previous example. Starting with $L = \{1, 2, 3, 4, 5, 6, 7\}$ (i.e. the seven given colors), $D_m = \{1, 6\}$ and $ND_m = \{2, 3, 4, 5, 7\}$, the Algorithm 2 tries to change each non-dominating color.

Initially, the **Color 7** is removed from the graph $G_{>0.15}$. For each vertex colored with 7 (i.e. **H** after **G**), the Algorithm 2 gives a new color for **H** and then for **G**. These new colors must be different from those of their neighborhoods. There are five possible colors (2,3,4,5,6) for **H** and also for **G** (1,2,3,4,5). According to formula (4), the chosen color for **H** is the one having the minimal distance with H, namely here **Color 5** (i.e. $D(\mathbf{H}, \mathbf{E}) = 0.05$ is the minimal and $c(\mathbf{E}) = 5$). For the vertex **G**, the chosen color is also the **Color 5** (i.e. $D(\mathbf{G}, \mathbf{H}) = 0.05$ is the minimal and $c(\mathbf{H}) = 5$). The Fig. 6 gives an illustration for the new colored graph.

Before starting again with another color $q' \in ND_m$, we verify if some colors of ND_m became a new dominating colors: it results that $D_m = 1, 6$ and $ND_m = 2, 3, 4, 5$.

Fig. 6 A new colored graph $G_{>0.15}$ (the color 7 is removed)

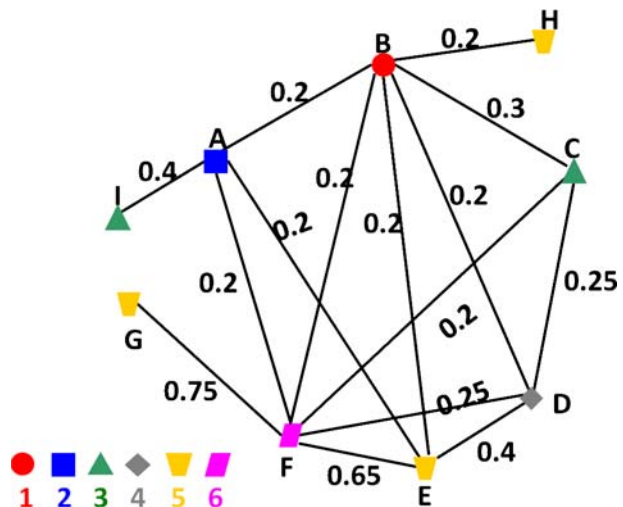
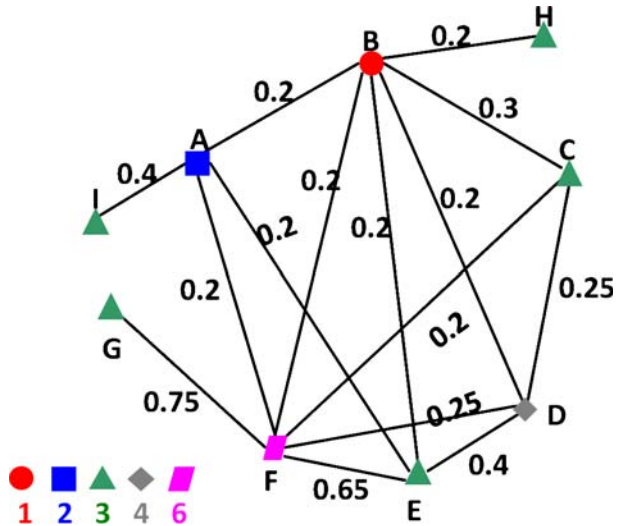


Fig. 7 A new colored graph $G_{>0.15}$ (the color 5 is removed)



Then another color of ND_m is taken (*i.e.* **Color 5**). The previous principle is applied to concerned vertices: **E**, **H** and **G**. The new color for **E** is 3 (*i.e.* the only possible color which is different from the colors of its neighborhood). The new color for **H** and **G** is also 3 (*cf.* Fig. 7). D_m becomes the set $\{1, 6, 3\}$ and ND_m the set $\{2, 4\}$.

Once again, another color of ND_m is taken (**Color 4**), which concerns only one vertex (**D**). The color of **D** is set to **Color 2** because it is the only color which is different from the colors of its neighborhoods. Thus, D_m becomes the set $1, 6, 3, 2$ and ND_m the empty set ϕ . Therefore, the b-coloring of the graph $G_{>0.15}$ given by Algorithm 2 is depicted in Fig. 8.

Fig. 8 The b-coloring of the graph $G_{>0.15}$. Four colors are identified

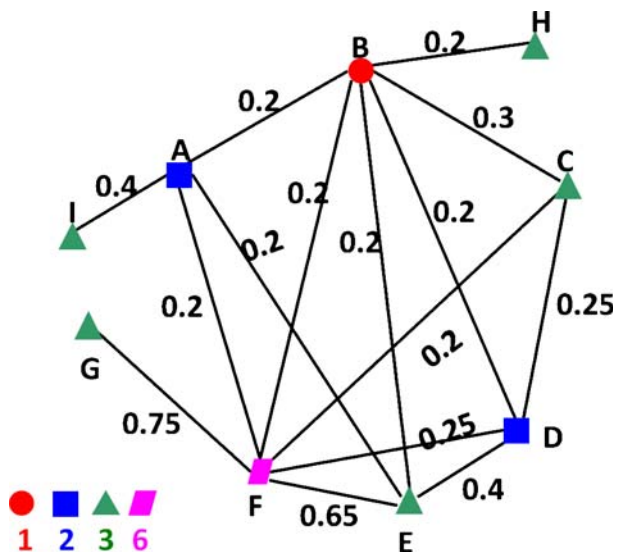
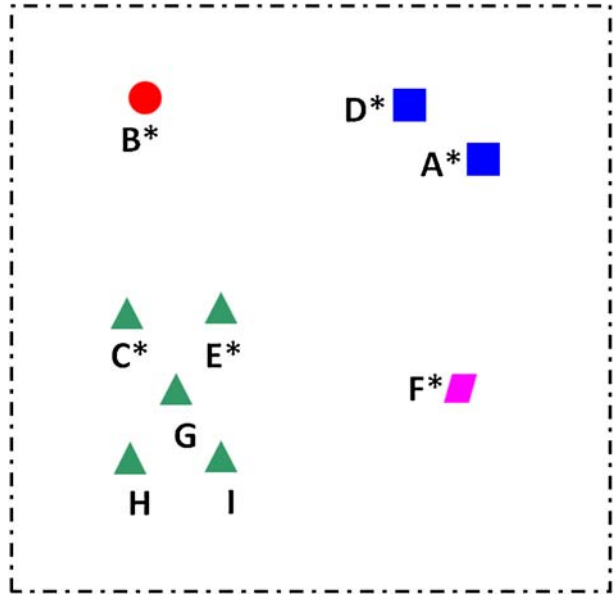


Fig. 9 The partition associated to $\theta = 0.15$. “*” is used to denote the dominating vertices



It appears that the graph $G_{>0.15}$ is colored using four colors, each color having at least one dominating vertex. Therefore, the algorithm associated with the threshold 0.15 splits the graph into the following classes: $C_1 = \{\mathbf{B}\}$, $C_2 = \{\mathbf{A}, \mathbf{D}\}$, $C_3 = \{\mathbf{C}, \mathbf{E}, \mathbf{G}, \mathbf{H}, \mathbf{I}\}$ and $C_4 = \{\mathbf{F}\}$ (Fig. 9). The vertices with the same color (shape) are grouped into the same cluster and the nodes with bold letter are the dominating vertices of classes. This means that these vertices are linked to at least one vertex in each other color class.

Proposition 5 *The algorithm gives a b-coloring of any graph G .*

Proof Suppose that the coloring of G is not a *b-coloring*. Therefore, the coloring of G is not a *proper coloring* or there exist at least one color c without any dominating vertex. Suppose that the coloring of G is not a *proper coloring*. This implies that there exist at least two adjacent vertices having the same color. However, as long as Algorithms 1 and 2 run, the color for every vertex is selected to being different from those of its neighbors, a contradiction. As Proposition 3 shows that every non-dominating color can be easily changed, Algorithm 2 reaches a fix point when the set of colors with no dominating vertices is an empty set and determines then a dominating coloring of G . This contradicts the assumption “there exists at least one color c in G without any dominating vertex”. Consequently, it is immediately obvious that using our algorithm, every graph should have a *b-coloring*. $\square$

Proposition 6 *Each cluster produced by the b-coloring based clustering algorithm has a connectivity diameter⁶ one in the complement of the graph $G(\mathbf{V}, \mathbf{E}_{>\theta})$.*

⁶The *connectivity diameter* of one cluster C is the longest distance between any two vertices in C , where the distance between two vertices is the minimum length of a path joining them.

Proof By the proper coloring property, we note that for each pair of adjacent vertices $(v_i, v_j) \in E_{>\theta}$, $c(v_i) \neq c(v_j)$. Hence, there are no edges between vertices in the same color and each cluster is then an *independent set*. By taking the complement of the graph G (i.e. the *inferior threshold graph* in which vertices correspond to elements and edges connect elements with dissimilarity values below the threshold θ), each cluster becomes a *highly connected subgraph* in which the vertices are connected by an edge. Consequently, the *connectivity diameter* of every cluster is one. $\square$

Remark Note that according to the formula (3), if the threshold graph G is a non-connected graph with connected components $C_1, C_2, \dots, C_p$, the algorithm is applied to each component C_i separately to find the *b-coloring* of G .

Discussion By Proposition 6, each cluster produced by the b-coloring based clustering algorithm has a *connectivity diameter* value of one in the complement graph. This is a strong indication to the *cluster homogeneity*, as the only better possibility in terms of the diameter is that each pair of vertices is connected by an edge within the same cluster.

To summarize, the first idea of the b-coloring based clustering algorithm is to start the graph coloring using a maximal number of clusters (i.e. colors). Then, each non-dominating colors are modified until all colors of G become dominating colors. With Algorithm 2, if there is a choice between many colors for vertex v_i , the closest color will be selected: this gives another strong indication of *homogeneity* and *clusters separation*.

Furthermore, there exists at least one *dominating vertex* for each color class which is linked (i.e., that is dissimilar) to at least one vertex in each other color class: such vertex reflects the properties of the cluster and also guarantees that the cluster has a *distinct separation* from other clusters of the partition. Our goal is not to obtain an optimal b-coloring but a good clustering based on graph b-coloring properties and performed in reasonable time.

5 Cluster Validation Stage

The clustering algorithm is iterative and performs multiple runs, each of them increasing the value of the dissimilarity threshold θ . The next problem is to validate the partitions of the data set and select the best one as the optimal output clustering, with regards to the number of clusters, as well as their separation and tightness values. In this section we proposed two different measures for between-cluster separation and intra-cluster homogeneity which can be used with symbolic data sets. Let $\{v_1, v_2, \dots, v_{\eta_i}\}$ be a set of η_i instances belonging to the cluster C_i ($\eta_i = |C_i|$ is the cardinality of the cluster C_i). Then the *average distance* $s_a(C_i)$ within the cluster C_i and the *between-cluster separation* $d_a(C_i, C_j)$ corresponding to the dissimilarity measures D is expressed as:

$$\begin{aligned} s_a(C_i) &= \frac{1}{\eta_i(\eta_i - 1)} \sum_{p=1}^{\eta_i} \sum_{q=1}^{\eta_i} d(v_p, v_q) \\ d_a(C_i, C_j) &= \frac{1}{\eta_i \eta_j} \sum_{p=1}^{\eta_i} \sum_{q=1}^{\eta_j} d(v_p, v_q) \end{aligned} \quad (5)$$

The values of cluster validity indices have been investigated in order to select the best partitioning among those obtained from our clustering algorithm. Different cluster validity indices have been proposed in the literature [27], which are adapted to the symbolic framework [28].

Therefore, we propose to define the best partition as the one with the dissimilarity threshold that optimizes an optimal index value.

Suppose a data set $V = \{v_1, v_2, \dots, v_n\}$ is clustered or grouped into a partition $P = \{C_1, C_2, \dots, C_k\}$ where for $\forall C_i, C_j \in P, C_i \cap C_j = \emptyset$ for $i \neq j$ and $\sum \eta_i = n$ such that η_i is the cardinality of the cluster C_i . We consider the following validity indices:

Davies-Bouldin Index is a function of the ratio of the sum of within-cluster scatter to between cluster separations. The best clustering minimizes the following value:

$$DB = \frac{1}{k} \sum_{i=1}^k \max_{j \neq i} \frac{s_a(C_i) + s_a(C_j)}{d_a(C_i, C_j)} \quad (6)$$

Generalized Dunn's Index was proposed to offer a compromise between the *inter-cluster separation* and the *intracluster cohesion* and recognized as a good assessment of the partition data set. So, it is the more appropriated to partition data set in *compact and well-separated* clusters. Basically we would like to maximize the *Dunn's generalized index* to achieve high quality clustering.

$$Dunn_G = \frac{\min_{i,j,i \neq j} d_a(C_i, C_j)}{\max_h s_a(C_h)} \quad (7)$$

where $C_h, C_i, C_j \in P$.

As an illustration, successive threshold graphs are constructed for each threshold θ selected from the dissimilarity Table 1, and our approach is used to give the *b-coloring partition* of each graph. The value of the *Generalized Dunn's index* is computed for the partitions obtained (cf. Table 2). Once all threshold values passed, we conclude that the optimal clustering which maximizes the *Dunn_G* index is the

Table 2 Evaluation of each threshold partition

Threshold	Partition	# clusters (k)	<i>Dunn_G</i>
0.05	{A}, {B}, {C}, {D}, {E,H}, {F}, {G}, {I}	8	1.50
0.075	{A}, {B,I}, {C}, {D}, {E,H}, {F}, {G}	7	1.00
0.10	{A}, {B,I}, {E,C}, {G,D}, {F,H}	5	1.25
0.15	{A,D}, {B}, {C,E,G,H,I}, {F}	4	1.52
0.20	{D,A,B,H}, {E,C,G,I}, {F}	3	1.16
0.25	{A,B,E,G,H}, {C,D,F,I}	2	1.15
0.30	{A,E,G,B,C,H}, {D,F,I}	2	1.30
0.40	{F,A,B,C,D,H,I}, {E,G}	2	1.32
0.65	{F,A,B,C,D,E,H,I}, {G}	2	1.01

one associated with the threshold $\theta = 0.15$. Thus, the optimal returned partition is $P = \{\{\mathbf{A}, \mathbf{D}\}, \{\mathbf{B}\}, \{\mathbf{C}, \mathbf{E}, \mathbf{G}, \mathbf{H}, \mathbf{I}\}, \{\mathbf{F}\}\}$.

6 Experiments on Benchmark Data Sets

The evaluation of clustering algorithms is always a challenge. The b-coloring clustering algorithm was tested by considering five relevant benchmark data sets, viz., *Zoo*, *Auto Import*, *Mushroom*, *Wine* and *Heart Disease* [29].

A comparison is made based on the quality of the partitioning obtained from our *b-coloring algorithm* and an assortment of six clustering approaches: the *Conceptual Clustering Algorithm* and the *Agglomerative Symbolic Clustering Algorithm*, both described in [28], the *Hansen's approach* which is based on the *minimal coloring technique* [19], the *Agglomerative Hierarchical Classification* [3] (in the form of *single-link*, *complete-link*, *average-link* and *centroid-based* approaches), the *Robust Clustering using Links Algorithm* (ROCK) [30], and the popular *k-means* clustering algorithm [7].

Despite their significant algorithmic differences, these clustering approaches share common properties with the b-coloring one. First the latter is based on pairwise dissimilarity matrix and can accommodate all types of data similarly as well as for the Agglomerative Hierarchical Classification and Hansen's approaches. Additionally, it is based on graph coloring likewise the Hansen's approach that used a minimal coloring problem. As the b-coloring algorithm is a partitioning algorithm, it will be compared to the k-means clustering algorithm.

Depending on the nature of attributes of the data set used, the choice of the dissimilarity function may vary. Thus the Euclidian distance can be applied to define the dissimilarity degree between two instances characterized with m features a_f ($f \in \{1 \dots m\}$). It is given by formula (1) where $\alpha = 2$ as follow:

$$d_{i,j} = D(v_i, v_j) = \left(\sum_{f=1}^m \left(\frac{g_f(a_{i,f}, a_{j,f})}{m_f} \right)^2 \right)^{\frac{1}{2}} \quad (8)$$

where m_f is the normalized coefficient for the attribute a_f , and g_f is the comparative dissimilarity function between the two attribute values $a_{i,f}$ and $a_{j,f}$ corresponding respectively to the instances v_i and v_j .

For *numeric* items:

$$\begin{aligned} g_f(a_{i,f}, a_{j,f}) &= |a_{i,f} - a_{j,f}| \\ m_f &= |\max_i a_{i,f} - \min_i a_{i,f}| \end{aligned} \quad (9)$$

For *categorical* or *Boolean* items:

$$\begin{aligned} g_f(a_{i,f}, a_{j,f}) &= \begin{cases} 1 & \text{iff } a_{i,f} \neq a_{j,f} \\ 0 & \text{iff } a_{i,f} = a_{j,f} \end{cases} \\ m_f &= 1 \end{aligned} \quad (10)$$

For an interesting assess of the results gained with the different clustering approaches, the following performance indices are used:

- Two *probability-matching* schemes called *cohesion* and *distinctness* [31] and related respectively to the *intracluster similarity* and the *intercluster dissimilarity*.
- A *label-matching* scheme called *Rand index* [32] which concerns the *clustering accuracy*. In the sequel, this index will be given for only labeled data sets.

6.1 Cohesion and Distinctness Functions

As used in [28], such functions are very powerful in the cluster validation problem because they are independent of both the number of clusters and the dissimilarity between objects. For a partition P with k clusters $\mathbf{P} = \{C_1, C_2, \dots, C_k\}$, the *cohesion* is used to reflect the compactness of the clusters. It is measured as the increased predictability of each attribute value of data set objects given the discovered partition in classes. The normalized increase in predictability M_{vh} , for an object v assigned to one class C_h , is defined as:

$$M_{vh} = \frac{1}{\sum_{i=1}^m |a_{v,i}|} \sum_{i=1 \dots m, j \in a_{v,i}} (\text{Prob}(a_i = V_{ij}|C_h)^2 - \text{Prob}(a_i = V_{ij})^2) \quad (11)$$

where m is the number of items describing one object. $|a_{v,i}|$ is the number of possible values for the attribute a_i of object v . $\text{Prob}(a_i = V_{ij})$ is the probability of attribute a_i to take the value V_{ij} . $\text{Prob}(a_i = V_{ij}|C_h)$ is the conditional probability of attribute a_i to take the value V_{ij} in class C_h .

The above equation assumes that a data object v has only one value per attribute (represented by $j \in a_{v,i}$).

The normalized value of the *cohesion* is expressed as the average of the M_{vh} values computed for all objects in the data set. This can be interpreted as the increase in match between a data object and its assigned cluster prototype over the match between the data object and the data set prototype [31].

$$\text{cohesion} = \frac{\sum_{h=1}^k \sum_{v \in C_h} M_{vh}}{n} \quad (12)$$

where n is the data set cardinality. The greater this value, the more cohesive are the clusters of partition $\mathbf{P}$.

The *distinctness* of $\mathbf{P}$ is defined as the intercluster dissimilarity using a probability match measure, namely the variance of the distribution match. The variance of the distribution match between clusters C_h and C_l in one partition is measured as:

$$\text{Var}(C_h, C_l) = \frac{1}{m} \sum_{i=1}^m \sum_{j \in a_i} (\text{Prob}(a_i = V_{ij}|C_h) - \text{Prob}(a_i = V_{ij}|C_l))^2 \quad (13)$$

where m is the number of attributes characterizing the objects. $\text{Prob}(a_i = V_{ij}|C_h)$ is defined in Eq. 11.

Table 3 Evaluation of zoo data clustering

Clustering approach	# clusters (k)	Cohesion	Distinctness
Proposed b-coloring approach	7	0.37	0.61
Agglomerative symbolic approach	4	0.27	0.58
Conceptual approach	2	0.08	0.50
Single-link approach	2	0.01	0.50
Complete-link approach	6	0.36	0.56
Average-link approach	8	0.37	0.59
Hansen's approach	4	0.28	0.55

The greater this value, the more dissimilar are the two clusters being compared, and, therefore, the meaning they express. The *distinctness* of the partition $\mathbf{P}$ is represented as the average variance between the partition clusters.

$$Distinctness = \frac{\sum_{h=1}^k \sum_{l=1}^k Var(C_h, C_l)}{k \times (k - 1)} \quad (14)$$

When comparing two partitions, the one that produces the greater *distinctness* should be preferred since the clusters in this partition better correspond to the distinct meanings or concepts used [31].

6.2 Clustering Accuracy Using Rand Index

In our case, some used UCI data set (viz., *Zoo*, *Wine* and *Heart Disease* [29]) includes *class information (label)* for each data instance. These labels are available for evaluation purposes but not visible to the clustering algorithm. Remember that the objective was to perform unsupervised classification that correctly identifies the underlying classes when the number of clusters is not predefined. Consequently, evaluation is based on a label matching scheme called *Rand index* which concerns *clustering accuracy*.

The partition obtained using one clustering approach will be assessed by considering relations upon instances: for each pair of instances, either they have the same label or different ones. For a data set with n instances, there are $n \times (n - 1)/2$ unique pairs of instances (v_i, v_j) , and thus there are $n \times (n - 1)/2$ pairwise decisions reflected in the obtained partition. The Rand index [32] is defined as:

$$Rand(\mathbf{P}, \mathbf{P}') = Accuracy = \frac{n_1 + n_2}{n \times (n - 1)/2} \quad (15)$$

Table 4 Accuracy performance on zoo data

Clustering approach	Accuracy
Proposed b-coloring approach	94.71%
Single-link approach	25.35%
Complete-link approach	92.44%
Average-link approach	93.21%
Hansen's approach	84.56%

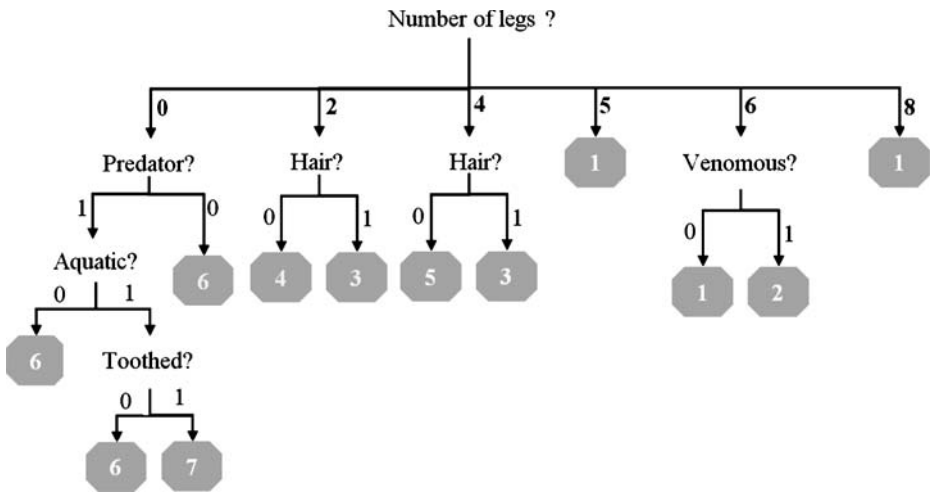


Fig. 10 Cluster representation using a classification tree for Zoo data set

where

- P is the correct partition given by the predefined class label.
- P' is the partition obtained with the clustering algorithm.
- n_1 and n_2 are the correct decisions. n_1 is the number of decisions where v_i and v_j are in the same cluster for partitions P and P' . n_2 is the number of decisions where v_i and v_j are placed in different cluster for P and P' .

When comparing two clustering algorithms, the one that produces the greater accuracy should be preferred since the partition correctly identifies the underlying classes in the data set.

Table 5 Clustering membership for zoo data

Cluster no	Animals
1	Crayfish; flea; gnat; housefly; ladybird; lobster; moth; octopus; scorpion; starfish; termite
2	Honeybee; wasp
3	Aardvark; antelope; bear; boar; buffalo; calf; cavy; cheetah; deer; elephant; fruitbat; giraffe; girl; goat; gorilla; hamster; hare; leopard; lion; lynx; mink; mole; mongoose; opossum; oryx; platypus; polecat; pony; puma; pussycat; raccoon; reindeer; sealion; squirrel; vampire; vole; wallaby; wolf
4	Chicken; crow; dove; duck; flamingo; gull; hawk; kiwi; lark; ostrich; parakeet; penguin; pheasant; rhea; skimmer; skua; sparrow; swan; vulture; wren
5	Crab; frog; newt; toad; tortoise; tuatara
6	Carp; clam; haddock; pitviper; seahorse; seawasp; slowworm; slug; sole; worm
7	Bass; catfish; chub; dogfish; dolphin; herring; pike; piranha; porpoise; seal; seasnake; stingray; tun

Table 6 Evaluation of auto import data clustering

Clustering approach	# clusters (k)	Cohesion	Distinctness
Proposed b-coloring approach	11	0.33	0.75
Agglomerative symbolic approach	3	0.11	0.39
Conceptual approach	2	0.07	0.33
Single-link approach	3	0.01	0.64
Complete-link approach	12	0.26	0.70
Average-link approach	11	0.27	0.72
Hansen's approach	8	0.21	0.49

6.3 Results

The clustering results over the benchmark data sets using the *Conceptual Clustering Algorithm* and the *Agglomerative Symbolic Algorithm* are given from [28]. Additionally, those using *ROCK* and the *Centroid-based approaches* are given from [30]. Other approaches have been implemented and for each of them, the *optimal partition* was returned as explained in Section 5: the correct partition of the data set is given by the smallest value of *Davis-Bouldin index*, while it corresponds to a large value of the *Generalized Dunn index*. For both indices, the evaluation metrics (*i.e. cohesion* and *distinctness*) of the resulting partition are provided. According to these values, the best partition is returned as an optimal clustering solution.

Zoo Data Set contains 100 instances of animals with 17 features and 7 output classes. The name of the animal constitutes the first attribute. There are 15 Boolean-valued attributes corresponding to the presence of hair, feathers, eggs, milk, backbone, fins, tail; and also airborne, aquatic, predator, toothed, breathes, venomous, domestic, catsize. The unique numeric attribute corresponds to the number of legs occurring in the set {0, 2, 4, 5, 6, 8}.

Table 3 provides the clustering results. Both *distinctness* and *cohesion* measures indicate better partitioning for 7 clusters generated by the proposed b-coloring clustering approach. The clusters are compact and well-separated. This confirms the pertinence of the b-coloring technique to offer a compromise between the *intercluster separation* and the *intracluster homogeneity*. Moreover, the b-coloring approach achieves better performances regarding the *clustering accuracy* (*c.f.* Table 4). The accuracy reaches 94.71% (25.35% for the *single-link*, 92.44% for *complete-link*, 93.21% for *average-link* and 84.56% for *Hansen's approach*).

Table 7 Evaluation of mushroom data clustering

Clustering approach	# clusters (k)	Cohesion	Distinctness
Proposed b-coloring approach	17	0.41	0.71
Agglomerative symbolic approach	10	0.02	0.64
Conceptual approach	4	0.01	0.57
Single-link approach	20	0.21	0.62
Complete-link approach	21	0.32	0.65
Average-link approach	22	0.34	0.67
Hansen's approach	19	0.36	0.68

Table 8 Clustering results for mushroom data

Cluster no	# of edible	# of poisonous	Cluster no	# of edible	# of poisonous
Centroid-based hierarchical clustering approach					
1	666	478	11	120	144
2	283	318	12	128	140
3	201	188	13	144	163
4	164	227	14	198	163
5	194	125	15	131	211
6	207	150	16	201	156
7	233	238	17	151	140
8	181	139	18	190	122
9	135	78	19	175	150
10	172	217	20	168	206
Rock					
1	96	0	12	48	0
2	0	256	13	0	288
3	704	0	14	192	0
4	96	0	15	32	72
5	768	0	16	0	1728
6	0	192	17	288	0
7	1728	0	18	0	8
8	0	32	19	192	0
9	0	1296	20	16	0
10	0	8	21	0	36
11	48	0			
Hansen's approach					
1	0	36	11	0	246
2	24	24	12	0	1296
3	16	8	13	96	248
4	192	0	14	0	1692
5	224	0	15	1728	0
6	24	8	16	512	150
7	288	0	17	192	50
8	96	42	18	48	32
9	48	84	19	48	0
10	672	0			
B-coloring approach					
1	0	36	11	139	0
2	96	464	12	18	0
3	695	72	13	0	1296
4	768	0	14	224	0
5	1510	0	15	0	1728
6	220	0	16	48	32
7	145	0	17	192	0
8	0	288			
9	144	0			
10	9	0			

Additional comparison is made from the results of the three agglomerative approaches (*single*, *complete* and *average*) when the same number of clusters is used as for the b-coloring method (i.e. 7 clusters, which is the number of the data set

Table 9 Evaluation of Heart Disease Data clustering

Clustering approach	# clusters (k)	Cohesion	Distinctness
Proposed b-coloring approach	8	0.14	0.68
Single-link approach	3	0.006	0.80
Complete-link approach	6	0.11	0.40
Average-link approach	2	0.003	0.61
Hansen's approach	3	0.05	0.45

underlying classes). The *cohesion* and *distinctness* for these approaches while giving 7 clusters are computed as: 0.26 and 0.66 (for *single-link*), 0.35 and 0.55 (for *complete-link*), 0.34 and 0.60 (for *average-link*), while the *accuracy* is estimated as 78.14% (for *single-link*), 92.72% (for *complete-link*) and 89.63% (for *average-link*) respectively.

These observations show that the b-coloring method gives also better results than the agglomerative approaches when the number of clusters is the same (except the single-link approach according to the *distinctness* value). The accuracy value indicates that the partition returned by the b-coloring approach correctly identifies the underlying classes in Zoo data set.

The optimal b-coloring partition with 7 clusters is enumerated in Fig. 10) and Table 5. It is observed that the resulting clusters are semantically meaningful on the basis of a number of features. Cluster 1 and 2 contain animals with high numbers of legs (≥ 5) while Cluster 2 consists of animals with six legs and can be distinguished from Cluster 1 on the basis of “venomous” attribute. On the other hand, Cluster 3 consists of furry animals with two or four legs while Cluster 4 and 5 contain animals with two and four legs respectively but without presence of hair. Finally, Cluster 6 and 7 consist of animals without legs although the Cluster 7 can be distinguished from Cluster 6 on the basis of “predator”, “aquatic” and “toothed” attributes. The evaluation of the latter cluster characteristics (c.f. Fig. 10) and memberships (c.f. Table 5) confirms that our approach gives more *meaningful* and *well-separated* clusters.

Auto Import Data Set consists of 193 instances of cars with 24 features. There are 14 quantitative (wheel-base, length, width, height, curb-weight, engine-size, bore, stroke, compression-ratio, horsepower, peak-rpm, city-mpg, highway-mpg, price) and 10 nominal (make, fuel-type, aspiration, number-of-doors, body-style, drive-wheels, engine-location, engine-type, number of cylinders, fuel-system) attributes. Each quantitative attribute is discretized to five quantiles.

Table 6 provides results of the Auto import data set clustering using different approaches. According to the *distinctness* and *cohesion* values, our b-coloring approach generates the best clustering.

In order to better assess the results obtained from the agglomerative approaches, *cohesion* and *distinctness* are evaluated when these approaches generate the same number of clusters that the b-coloring provides (i.e. 11 clusters). *Cohesion* and *distinctness* are computed as 0.06 and 0.85 for the *single-link*, 0.29 and 0.68 for *complete-link*. Excluding the *single-link* approach, the proposed b-coloring approach is better than the rest of methods. As expected, the single-link approach generates better results than ours according to the *distinctness* but fails on *cohesion*. This is due to the chaining effect problem related to this method. Indeed, it is observed that the

Table 10 Accuracy performance on heart disease data

Clustering approach	Accuracy
Proposed b-coloring approach	58.79%
Single-link approach	50.35%
Complete-link approach	56.19%
Average-link approach	50.18%
Hansen's approach	53.36%

cluster size significantly varies (one cluster among the 11 generated by the single-link method contains 179 among the 193 instances in the data set).

Mushroom Import Data Set was also obtained from the UCI Machine Learning Repository. Each data record contains information that describes the 21 physical properties (*e.g.*, color, odor, size, shape) of a single mushroom. A record also contains a poisonous or edible label for the mushroom. All attributes are categorical; for instance, the possible values for “size” attribute are: narrow, broad, while the values of “shape” can be bell, at, conical, convex, and “odor” is one of spicy, almond, foul, fishy, pungent, etc. The mushroom database contains 8124 records and is the largest data set we used in our experiments. The number of *edible* and *poisonous* mushrooms in the data set is 4208 and 3916, respectively. There are 23 species of mushrooms in this data set. Each species is then identified as definitely edible, definitely poisonous, or of unknown edibility and not recommended to eat. Therefore the latter class is combined with the poisonous one.

Table 7 provides the results of the clustering obtained on the mushroom data using the same panel of clustering approaches: *b-coloring*, *conceptual*, *Agglomerative symbolic*, *Hansen's* and the (*single-link*, *complete-link* and *average-link*) *Agglomerative Hierarchical*.

Moreover, Table 8⁷ shows the membership differences between clusters that were produced over the mushroom database by the (1) *proposed b-coloring algorithm* (17 clusters), (2) *Hansen's algorithm* (19 clusters), (3) *centroid-based hierarchical clustering algorithm* (20 clusters) and (4) *ROCK* (21 clusters). From the results in Table 8, we observe that:

- All the clusters identified by ROCK except one (Cluster 15) are pure clusters: each cluster includes either all poisonous or all edible mushrooms.
- On the other hand, all the clusters discovered by the centroid-based hierarchical clustering algorithm are *non pure* clusters.
- All the clusters discovered by Hansen's approach except nine are pure clusters.
- All the clusters discovered by our approach except three (Cluster 2, Cluster 3 and Cluster 16) are *pure* clusters.

Discussion First, it can be noticed that the clustering quality of the hierarchical algorithm is poor. Every cluster from this partition contains a sizeable number of both poisonous and edible mushrooms, and the cluster size is fairly uniform. Thus, more than 90% of cluster size is between 200 and 400, and only one cluster has more than 1000 mushrooms. These results are expected due to the problem of single-point

⁷The data in italics represents *non-pure clusters* (including both poisonous and edible mushrooms).

Table 11 Evaluation of wine data clustering

Clustering approach	# clusters (k)	Cohesion	Distinctness
Proposed b-coloring approach	4	0.14	0.37
Hansen's approach	4	0.11	0.34
k-means algorithm	5	0.08	0.33

representation of clusters mentioned in Section 2: *centroid-based method* cannot easily capture the clusters with a large variation in the size. It can be concluded that in this partition, *the clusters are not well-separated*.

The best results are given with ROCK algorithm which is limited to categorical attributes but is presented as a robust clustering algorithm for such data [30]. Our algorithm, which has no limitation on attribute types, returns approximately similar results than ROCK ones, and best results than Hansen's approach (which also uses the minimal coloring technique). This confirms that the concept of dominating vertex allow more *meaningful* and *well-separated* clusters to be issued.

Heart Disease Data Set is very interesting due to its real and mixture appearance. It consists of 303 instances of heart disease (generated at Cleveland Clinic on 1988) with 13 features. There are 5 numeric and 8 categorical attributes. All instances were also classified into two classes: healthy (buff) or with heart-disease (sick).

The clustering results are provided in Table 9.⁸ Once again according to the *cohesion* and *distinctness* measures, the b-coloring approach generates the best clustering. As previously observed, the *single-link* approach generates better results than ours according to the *distinctness* but fails on both *cohesion* and *accuracy* (c.f. Table 10). This is due to the fact that the clusters are not well-constructed (the well-known chaining effect). In fact, it is observed that the cluster size varies considerably (one cluster among the three generated by the *single-link* method contains 300 instances).

Furthermore, the Agglomerative approaches do not reach the results given from the b-coloring even when using the same number of clusters (i.e. 8 clusters). *Cohesion* and *distinctness* for the three cases are computed as 0.02, 0.88 (*single-link*), 0.13, 0.41 (*complete-link*), and 0.1, 0.63 (*average-link*) respectively.

The clustering *accuracy* climbs to 58.79% for the graph b-coloring clustering algorithm. Otherwise, the rest of algorithms (i.e. *single-link*, *complete-link*, *average-link* and *Hansen's*) achieve respectively an *accuracy* of 50.35%, 56.19%, 50.18% and 53.36% (c.f. Table 10).

Moreover, our algorithm produces best results than the *single-link*, *complete-link* and *average-link* approaches when the number of clusters is fixed at 8 (i.e. the same number of clusters selected from the b-coloring approach). They achieve an *accuracy* of 50.56 %, 54.86% and 54.86% respectively.

Wine Data Set Previous experiments were based on data sets using categorical attributes and various clustering approaches including hierarchical ones. Additional experiments were conducted using the popular *k-means* partitioning algorithm [7]

⁸In order to compute the *Cohesion* and *Distinctness*, each numeric attribute is discretized to five quantiles.

Table 12 Accuracy performance on wine data

Clustering approach	Accuracy
Proposed b-coloring approach	83.75%
Hansen's approach	74.78%
k-means algorithm	69.15%

compared with our b-coloring clustering approach on one data set composed of numeric (continuous) attributes.

The Wine data set contains the results of a chemical analysis of wines grown in a specific area of Italy. Three types of wine are represented in the 178 samples, with the results of 13 chemical analyses (numeric values) recorded for each sample.

The clustering results are provided in Tables 11⁹ and 12. According to the *cohesion*, *distinctness* and *accuracy* measures, once again it's the b-coloring approach which generates the best clustering. Using $k=4$ as an input parameter for *k-means*, the *cohesion* and *distinctness* are computed as 0.09, 0.37 and the *accuracy* is estimated to 70.17%.

Notice Throughout the experiments conducted on benchmark data sets, the optimal partition (according to the *cohesion*, *distinctness* and *accuracy* measures) actually corresponded to the greater value of Generalized Dunn's index. This confirm the idea that this index is a good assessment of clustering quality and that it is well-suited to partition data set in compact and well-separated clusters.

7 Experiments on Real Data Set

In French hospitals, the Diagnosis Related Groups (DRG) Casemix system was introduced in the eighties with a new medical information system provision called PMSI. PMSI data are related to the surgical/medical treatments, medical exams and diagnoses, as well as patient's personal characteristics such as age, sex, a medical gravity rating, the number of visited medical units, the length of stay in the intensive care unit, etc. Predefined rules govern the classification of each hospital stay of one patient into one of approximately 800 disease groups (DRG) using a national software program. Each disease group is intended to be homogeneous in medical category and resource consumption, and notably each hospital stay of the same DRG group should run the same number of days. The DRG groups therefore play a considerable role in hospital budget allocation.

In spite of the grouper and DRG budgets are reviewed each two years by the national French Hospital Information Technical Agency (ATIH), which modify them on the basis of cost data gathered from a few selected referent hospitals, the DRG groups remain inefficient for both public and private hospitals. The major problem is that very different pathologies and examinations appear within the same DRG, which makes both medical professionals and hospital managers unsatisfied. For example considering the length of the stay, some hospital centers could provide

⁹In order to compute the *Cohesion* and *Distinctness*, each numeric attribute is discretized to five quantiles.

appropriate health care shorter than what is planned for the DRG-group. On the opposite, they cannot sometimes achieve patient care in the planned conditions without running a high medical risk for the patient. The consequence is that opportunist behaviors appear when filling out PMSI records or in patient selection. Therefore both administrators (who know that clinicians sometimes don't give the right PMSI code) and clinicians have little confidence in the DRG groups.

In this section, the clustering algorithm described in Section 4 has been investigated to provide an alternative for the DRG groups. The fact is that the number of groups is not known in advance and no matter if we obtain more/less classes than the 800 existing DRG groups. Administrator and clinician are looking for hospital stay groups with a high cohesion (both on medical and economic points of view) within each group, and well-separated classes. They found interest in our approach essentially because we proposed that both medical and economic aspects (which are represented in PMSI data) were taken into account in the designing of new hospital stay groups. The algorithm was tested on a large sample of PMSI data to build a refined typology of *hospital stays* in a retrospective way. In the same time, the homogeneity of current DRG groups has been formally analyzed.

The data set consists of 2750 instances of hospital stays, which actually correspond to 79 different existing DRG groups (many stays were assigned to the same DRG group). In order to guarantee the original DRGs distribution within the sample, a sampling procedure called *proportionate stratified sampling* was applied. The description of PMSI hospital stays is based on 7 items: 2 quantitative items, viz., *length of the stay* and *number of surgical/medical treatments*, 2 qualitative items, viz., *output mode of the stay* and *patient's age* (discretized into four medical quantiles), 1 Boolean item, viz., *presence/absence of "medical associated diagnosis"*, 2 symbolic attributes, viz., *the principal medical diagnosis and medical or surgical treatments*.

Let $V = \{v_1, v_2, \dots, v_n\}$ be the *set of hospital stays* to be clustered. Each hospital stay is then represented by a mixture of 7 symbolic and/or classical attributes denoted a_f ($f \in \{1..7\}$). Therefore, the hospital stay v_i is described by one 7-dimensional vector $(a_{i,2}, a_{i,2}, \dots, a_{i,7})$.

Similarly to the previous experiments with benchmark data sets, the Euclidian distance (Eq. 8) was applied to define the dissimilarity measure between hospital stays. The comparative dissimilarity function gf between two quantitative/qualitative attribute values remains the same as in formula (9)/(10). In the case of symbolic items (i.e. here, items described by a set of values), we proposed to investigate the data organization. Indeed, medical diagnosis of diseases and injuries are coded in the International Classification of Diseases, 10th revision (ICD-10), and the French Catalogue of Medical Acts (CDAM) is used for medical and surgical treatments. Items within these classifications are both hierarchically organized.

For example, the CDM is currently organized into five fields, each field corresponding to one medico-technical field. Fields are divided into chapters, which are subdivided in sub-chapters, which are also cut in paragraphs, then in subparagraphs, and finally in headings.

In such a structure, the deeper you go along the same branch (i.e. headings), the more similar treatments become. At high level nodes, the dissimilarity between treatments and acts is higher (i.e. at the field level).

We found that the *Hausdorff dissimilarity* was the most suitable distance to capture the hierarchical similarity property. Named after Felix Hausdorff, the *hausdorff*

Table 13 Examples of medical treatment description

Treatment	Description
P035	Electronic monitoring of the uterine contractions and the cardiac rate.
P045	Forceps delivery or spatula in cephalic presentation.
P060	Artificial delivery: manual extraction of the complete placenta.
P080	Immediate suture of simple perineal laceration, vaginal laceration or vulvar laceration.
P088	Episiotomy repair.

dissimilarity is the “maximum dissimilarity of a set to the nearest point in the other set” [33]. More formally, the *hausdorff dissimilarity* from a set $a_{i,f}$ to a set $a_{j,f}$ is a *max-min function*, defined as:

$$g_f(a_{i,f}, a_{j,f}) = \max \left\{ \sup_{x \in a_{i,f}} \inf_{y \in a_{j,f}} d(x, y), \sup_{y \in a_{j,f}} \inf_{x \in a_{i,f}} d(x, y) \right\} \quad (16)$$

$$m_f = \max_{x,y} (d(x, y)) \quad \forall x, y \in \text{hierarchy}$$

where $d(x, y)$ is the dissimilarity between x and y . In order to compare two medical treatments (respectively two diagnosis codes), the distance $d(x, y)$ is defined as the number of edges separating the treatment nodes x and y (respectively diagnosis nodes) in the hierarchical structure of CDAM (respectively ICD-10).

The previous formula means we calculate the dissimilarity of the first element from $a_{i,f}$ with all the elements from $a_{j,f}$, and get the minimum value, and so on for all the other elements composing $a_{i,f}$. We keep the maximum of these minimum values. We proceed the same way to determine the distance from $a_{j,f}$ to $a_{i,f}$ and we finally get the greater value between the two results.

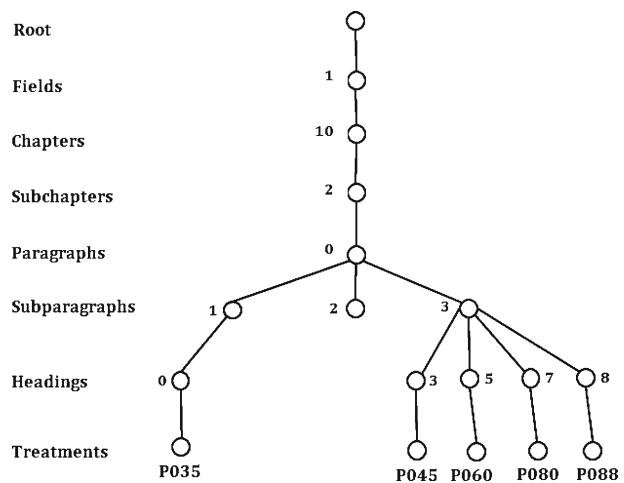
However, this method has an annoying issue due to the presence of outliers, considering the maximum value results in having too great dissimilarities between documents that are supposed to be similar. Consequently, in order to simultaneously deal with the semantic and structural properties of CDAM (respectively ICD-10) structure, we have proposed to alter the Hausdorff dissimilarity as follows:

$$g_f(a_{i,f}, a_{j,f}) = \max \left\{ \frac{\sum_{x \in a_{i,f}} \inf_{y \in a_{j,f}} d(x, y)}{|a_{i,f}|}, \frac{\sum_{y \in a_{j,f}} \inf_{x \in a_{i,f}} d(x, y)}{|a_{j,f}|} \right\} \quad (17)$$

Instead of considering the maximum as in the previous formula, we ought to use the average of the minimum values sum. An example is given by comparing two

Table 14 Location of the medical treatments in the CDAM structure

Treatment	P035	P045	P060	P080	P088
Field	1	1	1	1	1
Chapter	10	10	10	10	10
Sub-chapter	2	2	2	2	2
Paragraph	0	0	0	0	0
Sub-paragraph	1	3	3	3	3
Heading	0	3	5	7	8

Fig. 11 Part of the hierarchy structure of CDAM

hospital stays having respectively the treatment sets {P035, P088} and {P045, P060, P080}, which are described in Tables 13 and 14 and shown in Fig. 11. For example, the distance d between treatments P060 and P088 is calculated as the number of edges separating the two treatment nodes, which is equal to 4 as observed in Fig. 11. Therefore, the modified Hausdorff distance between the treatment sets {P035, P088} and {P045, P060, P080} is set to 5.

As mentioned earlier, the intended objective is to generate maximally cohesive clusters (maximize *intracluster similarity*) while achieving maximum separability among clusters (*intercluster dissimilarity*). Here each cluster represents a new group of hospital stays (the new DRGs). For performance goals, the *b-coloring clustering* results were compared with that of *Single-linkage Agglomerative Hierarchical Classification* (AHC) [3], the *Hansen's approach* based on *minimal coloring* technique [19] and the existing *DRG classification*.

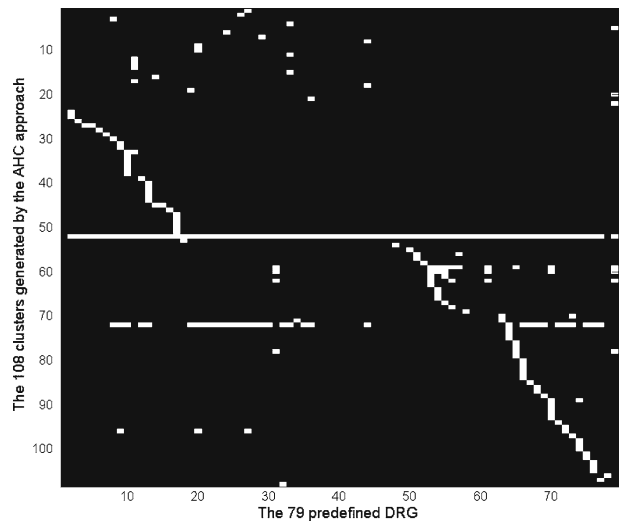
Table 15 provides the results of clustering obtained over the PMSI data sample using these four approaches. For the *b-coloring clustering* approach, which provides multiple partitions based on the dissimilarity threshold values θ , the partition which gives the larger value of *Generalized Dunn's index* were selected as the best partition.

Discussion According to the values of *cohesion*, *distinctness* and *Dunn_G index*, our clustering approach gives a better partitioning than the original DRG classification

Table 15 Evaluation of PMSI data clustering

Clustering approach	# Clusters (k)	$Dunn_G$	Cohesion	Distinctness
Proposed b-coloring approach	107	0.796	0.163	0.394
Hansen's approach	101	0.772	0.128	0.381
Single-link approach	108	0.766	0.045	0.572
Existing DRG classification	79	0.616	0.149	0.360

Fig. 12 The block-diagonal matrix DRG (79) * Generated AHC clusters (108)



and also than Hansen's approach. As expected, the *Single-link agglomerative approach* (AHC) generates better results than ours according to the *distinctness* but fails on both *cohesion* and *Dunn_G* index index (even lower than the DRG classification). This is because clusters are not well-constructed (as mentioned earlier, the method suffers from chaining effect) and cluster size varies considerably (one cluster among the 108 generated by the AHC method contains more than 1900 hospital stays, while more than 50% of the partition are singleton clusters).

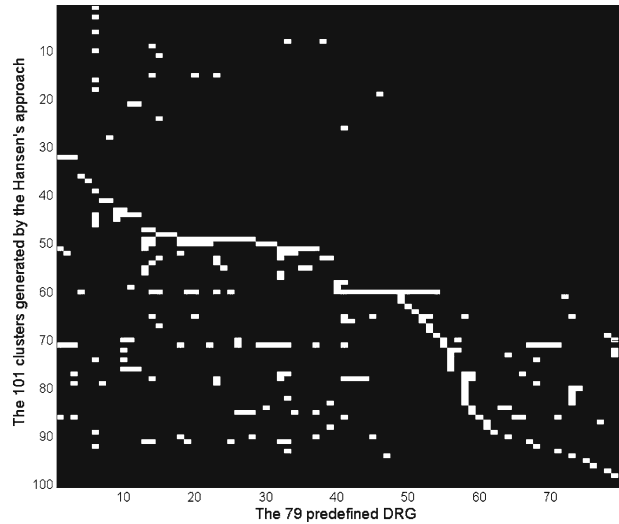
Furthermore, Figs. 12, 13 and 14 show the distribution (i.e. *block-diagonal matrix*¹⁰ also named two-cell arrangement) of the 79 DRG in the respectively 108, 101 and 107 clusters generated by the proposed AHC approach, Hansen's Approach and b-coloring clustering approach, respectively.

These matrices provide a visual representation of the accuracy of clustering methods to identify underlying hospital stay groups. The two-cell arrangement which contains a minimum number of exceptional elements (i.e. white cases outside the diagonal block, also named noise) shows the clustering approach ability to discover more *meaningful* and *well-separated* groups. In fact exceptional elements represent clusters which include many different predefined DRGs. The more numerous are exceptional points for one clustering method, the more the clusters include various DRGs and higher becomes the cluster heterogeneity.

It can be observed from the figures that a large number (among the 79 DRGs) are subdivided on two clusters at least. This confirms the heterogeneity problem of DRGs (i.e. very different diseases and examinations sometimes appear within the same DRG). As expected, the AHC block-diagonal matrix (cf. Fig. 12) includes one group with a large variety of hospital stays (many DRGs are represented in this

¹⁰In the *block-diagonal matrix*, one case (corresponding to one DRG class i in line and one hospital stay group j in column) have a white (resp. black) color if the group j contains hospital stays pre-classified on the DRG class i .

Fig. 13 The block-diagonal matrix DRG (79) * Generated Hansen's clusters (101)

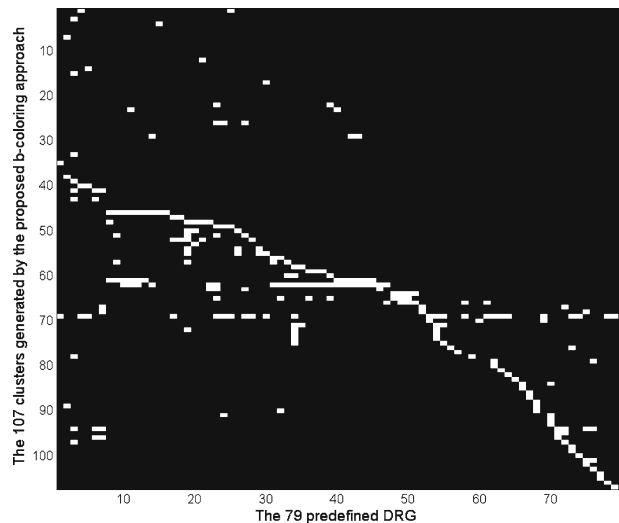


group: it is the cluster mentioned above which has more than 1900 hospital stays): *it isn't a cohesive cluster*.

Figure 14 shows that the b-coloring block-diagonal matrix provides less exceptional elements than the Hansen's block diagonal matrix (*cf.* Fig. 13). More often, DRGs are split in several sub-groups, which makes more cohesive groups.

A validation stage has also been performed with specialists from the medical domain (surgeons and institutional people) for advice about the medical meaningful of clusters. They have actually recognized the medical validity of the hospital stay groups obtained with our method, mentioning that many of the new groups actually corresponded to changes they intended to perform on actual DRGs.

Fig. 14 The block-diagonal matrix DRG (79) * Generated b-coloring clusters (107)



8 Conclusion

In this paper, a new clustering algorithm was proposed. It is based on a graph b-coloring technique. The building of the b-coloring of a specified graph is given in two stages: 1) initializing the colors of vertices with maximal colors, and 2) removing, by a greedy procedure, the colors without any dominating vertex. Subsequently, a number of cluster validity indices were proposed in order to determine the optimal partition according to specific constraints related to graph properties. These indices were designed to identify a partition of data in *compact* and *well separated* clusters.

We have implemented, performed experiments, and compared our method to other clustering approaches, both traditional and graph based approaches. We have also illustrated the method efficiency on relevant benchmark data sets, as well as on real medical (*PMSI*) data sets. It can be concluded that the main advantage of the proposed technique is gain from the graph-based technique used, which offers a real representation of clusters by *dominant objects* that guarantees to find *meaningful* and *well-separated* clusters of the data set.

Many interesting issues were raised from this work and some are currently being developed: (1) building a learning algorithm for automatic classification of new data arrivals, (2) exploiting the property of *color dominance* for clinical pathway discovery, and (3) defining some operators which permit to combine easily different clusterings constructing on different data, to name a few.

Acknowledgements The work described in this paper was supported by the Rhône-Alpes area council under the *DOMECAD project*. The authors gratefully acknowledge the contributions of *Mrs Bourgoïn* in providing the *PMSI data set*.

We would especially like to thank the anonymous reviewers of this paper for their assistance and insightful suggestions on the work as it progressed.

References

1. Ichino, M., Yaguchi, H.: General minkowsky metrics for mixed feature type data analysis. *IEEE Trans. Syst. Man Cybern.* **24**, 698–708 (1994)
2. Malerba, D., Esposito, F., Gioviale, V., Tamma, V.: Comparing dissimilarity measures in symbolic data analysis. In: Joint Conferences on New Techniques and Technologies for Statistics and Exchange of Technology and Know-how, pp. 473–481, Hersionissos, 18–22 June 2001
3. Jain, A.K., Murty, M.N., Flynn, T.P.J.: Data clustering: a review. *ACM Comput. Surv.* **31**, 264–323 (1999)
4. Guha, S., Rastogi, R., Shim, K.: Cure: an efficient clustering algorithm for large databases. In: Proceedings of the ACM SIGMOD Conference, pp. 73–84, Seattle, 2–4 June 1998
5. Karypis, G., Han, E., Chameleon, K.V.: A hierarchical clustering algorithm using dynamic modeling. *IEEE Comput.* **32**(8), 68–75 (1999)
6. Ng, R., Han, J.: Clarans: a method for clustering objects for spatial data mining. *IEEE Trans. Knowl. Data Eng.* **14**(5), 1003–1016 (2002)
7. Hartigan, J., Wong, M.: Algorithm as136: a k-means clustering algorithm. *J. Appl. Stat.* **28**, 100–108 (1979)
8. Auguston, J.G., Minker, J.: An analysis of some graph theoretical clustering techniques. *J. Assoc. Comput. Mach.* **17**(4), 571–588 (1970)
9. Zahn, C.T.: Graph-theoretical methods for detecting and describing gestalt clusters. *IEEE Trans. Comput.* **20**, 68–86 (1971)
10. Matula, D.W.: Cluster analysis via graph theoretic techniques. In: Mullin, R.C., Reid, K.B., Roselle, D.P. (eds.) *The Louisiana Conference on Combinatorics, Graph Theory, and Computing*, pp. 199–212. Louisiana State University, Baton Rouge (1970)

11. Matula, D.W.: K-components, clusters, and slicings in graphs. *SIAM J. Appl. Math.* **22**, 459–480 (1972)
12. Shi, J., Malik, J.: Normalized cuts and image segmentation. *IEEE Trans. Pattern Anal. Mach. Intell.* **22**(8), 888–905 (2000)
13. Wu, Z., Leahy, R.: An optimal graph theoretic approach to data clustering: theory and its application to image segmentation. *IEEE Trans. Pattern Anal. Mach. Intell.* **15**(11), 1101–1113 (1993)
14. Gotlieb, C.C., Kumar, S.: Semantic clustering of index terms. *J. Assoc. Comput. Mach.* **15**(4), 493–513 (1968)
15. Guenoche, A., Hansen, P., Jaumard, B.: Efficient algorithms for divisive hierarchical clustering with the diameter criterion. *J. Classif.* **8**, 5–30 (1991)
16. Hartuv, E., Shamir, R.: A clustering algorithm based on graph connectivity. *Inf. Process. Lett.* **76**, 175–181 (2000)
17. Gomory, R.E., Hu, T.C.: Multy-terminal network flows. *SIAM J. Appl. Math.* **9**, 551–560 (1961)
18. Kuhns, J.L.: Mathematical analysis of correlation clusters. *Word correlation and automatic indexing*, 2 (1959)
19. Hansen, P., Delattre, M.: Complete-link cluster analysis by graph coloring. *J. Am. Stat. Assoc.* **73**, 397–403 (1978)
20. Elghazel, H., Kheddouci, H., Deslandres, V., Dussauchoy, A.: A partially dynamic clustering algorithm for data insertion and removal. In: Corruble, V., Takeda, M., Suzuki, E. (eds.) *DS2007* (Springer Verlag LNAI 4755), pp. 78–90. Springer, New York (2007)
21. Irving, W., Manlov, D.F.: The b-chromatic number of a graph. *Discrete Appl. Math.* **91**, 127–141 (1999)
22. Effantin, B., Kheddouci, H.: The b-chromatic number of some power graphs. *Discret. Math. Theor. Comput. Sci.* **6**(1), 45–54 (2003)
23. Kouider, M., Maheo, M.: Some bounds for the b-chromatic number of a graph. *Discret. Math.* **256**, 267–277 (2002)
24. Effantin, B., Kheddouci, H.: A distributed algorithm for a b-coloring of a graph. In: *ISPA2006* (Springer Verlag LNCS 4330), pp. 430–438. Springer, New York (2006)
25. Kheddouci, H.: *Placement et paramètres de graphes*. Mémoire HDR RC 20586, University of Bourgogne (2003)
26. Elghazel, H., Deslandres, V., Hacid, M.S., Dussauchoy, A., Kheddouci, H.: A new clustering approach for symbolic data and its validation: application to the healthcare data. In: Esposito, F., Ras, Z.W., Malerba, D., Semeraro, G. (eds.) *ISMIS2006* (Springer Verlag LNAI 4203), pp. 473–482. Springer, New York (2006)
27. Bezdek, J.C., Pal, N.R.: Some new indexes of cluster validity. *IEEE Trans. Syst. Man Cybern.* **28**(3), 301–315 (1998)
28. Kalyani, M., Sushmita, M.: Clustering and its validation in a symbolic framework. *Pattern Recogn. Lett.* **24**(14), 2367–2376 (2003)
29. Blake, C.L., Merz, C.J.: *Uci Repository of Machine Learning Databases*. University of California Irvine, Irvine (1998)
30. Guha, S., Rastogi, R., Shim, K., Rock: A robust clustering algorithm for categorical attributes. *Inf. Syst.* **25**(5), 345–366 (2000)
31. Biswas, G., Weinberg, J.B., Fisher, D.H.: Iterate: a conceptual clustering algorithm for data mining. *IEEE Trans. Syst. Man Cybern.* **28**(C), 219–230 (1998)
32. Rand, W.M.: Objective criteria for the evaluation of clustering methods. *J. Am. Stat. Assoc.* **66**, 846–850 (1971)
33. Rote, G.: Computing the minimum Hausdorff distance between two point sets on a line under translation. *Inf. Process. Lett.* **38**, 123–127 (1991)